AI Forward Deployed Engineer

The AI Architect & FDE Course



3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- [x] Describe the role of the AI Forward Deployed Engineer
- [x] Define an agent and a harness; build an agent with LangChain
- [x] Compare proprietary and open-weight models
- [x] Run inference on local models

By the end of today, you'll be able to:

- [ ] Explain what's involved in Harness Engineering
- [ ] Describe the landscape of frameworks for building a Harness
- [ ] Apply Claude Agent SDK & Deep Agents to a task
- [ ] Deliver an agent harness for a business goal

Recap

Open-source / open-weight, local and token cost on Day 4



Proprietary and open-weight models
Pros & cons; open-weight not necessarily cheaper & more private



Inside the models
Dense vs Mixture of Experts models, Quantization, Distillation



Running models
Local engines like Ollama and hosted providers on OpenRouter



Token cost
Stay focused on the cost to deliver your business task

Today we return to the pilot and build a harness for its agent

The evolution

The skill everyone is looking for in AI has changed from 2024 - 2026

1



Prompt Engineering

Construct the message to the LLM

2



Context Engineering

Construct the input to the Agent

3



Harness Engineering

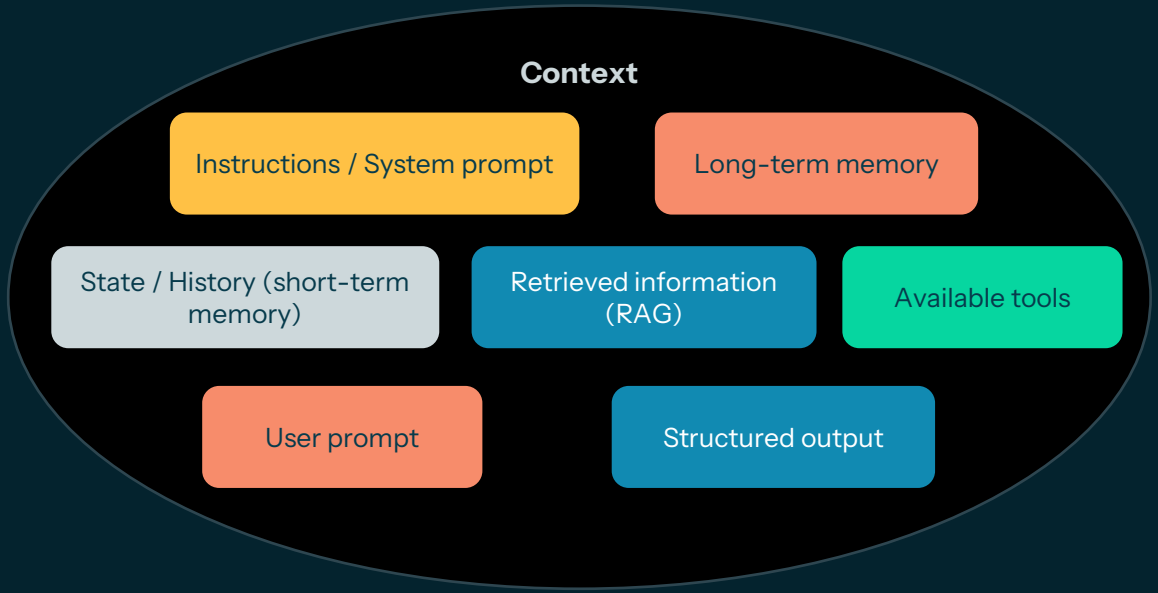
Construct the scaffolding for the Agent or Agents

Harness Engineering can be used in the context of working with Agents (like Claude Code)...
...or building Agents like the UnderCover Agent

Context Engineering

"The New Skill in AI is Not Prompting, It's Context Engineering"

Philipp Schmid, Google DeepMind



Defining harness engineering

"...anytime you find an agent makes a mistake, you take the time to engineer a solution such that the agent never makes that mistake again"

Mitchell Hashimoto, co-founder of HashiCorp, February 2026

"Humans steer. Agents execute."

Ryan Lopopolo, OpenAI, in the blog post "Harness engineering", February 2026

"Agent = Model + Harness. If you're not the model, you're the harness."

Vivek Trivedy, LangChain, March 2026

Harness engineering is..

Building the scaffolding around LLMs
...so agents can take on increasingly impactful tasks
...and deliver them reliably, safely and successfully
...on their own, under your leadership

Harness engineering includes

What makes up an Agent Harness



Context
Prompts,
tools, memory



Workflow
The loop
the agent runs



State
Carried
between steps



Sub-agents
Delegation
& teams



Permissions
Sandboxes
and approvals



Observability
Traces
and evaluations



Self-improvement
And recursive
self-improvement

With harness frameworks, some comes out of the box; the rest you configure or build

Harness User vs Harness Builder

Two completely different ways you could apply “Harness Engineering”

As a user of Agents: allow them to do more

The agent
Claude Code, building UnderCover for us

Its harness
AGENTS.md, the docs, hooks and tests

On the course
Day 3 and each product change after it

As a builder of Agents: build them to do more

The agent
The UnderCover Agent deciding claims

Its harness
The loop, tools, to-dos and subagents

On the course
Today and through Weeks 2 and 3

The primary focus of this course

Harness engineering UnderCover

Ways we can improve the agent to drive further value



An end goal
Every claim in the queue ends with a decision



Tools to find its own work and information
The waiting claims, the policy wording and the claims manual



A to-do list
It plans the queue and ticks off each claim as it goes



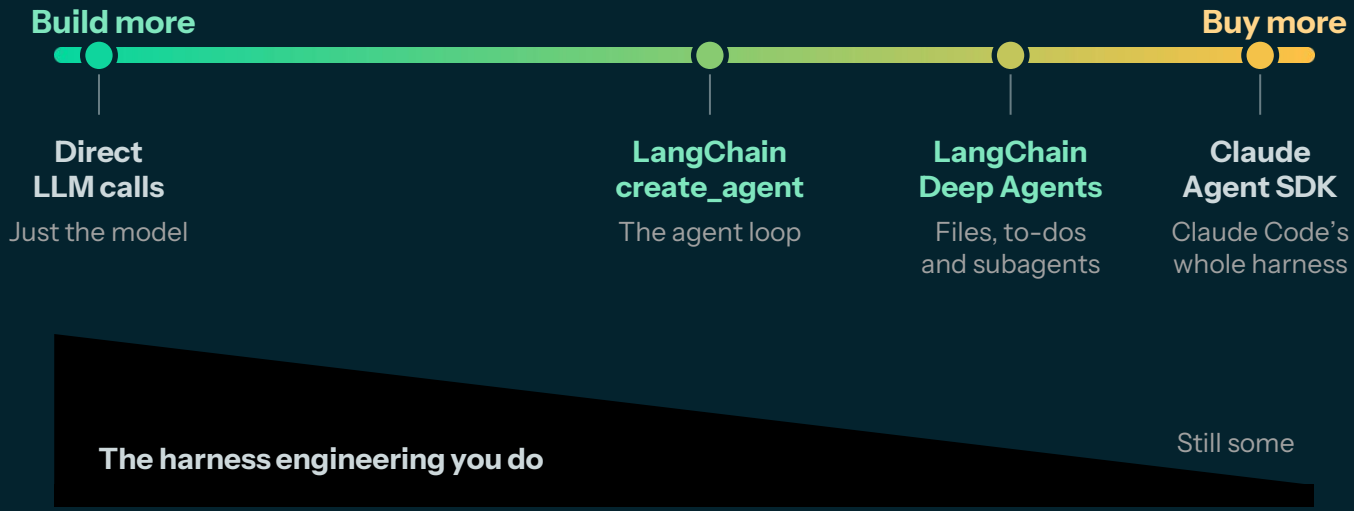
A fresh context for each piece of work
A subagent works one claim and reports back in one line



A way to recover and hand back
Failed claims are tried again; unsure claims go to a handler

Build or buy?

A harness engineer picks a framework and extends it



Today we try both frameworks in the labs, then extend create_agent in the product

Today’s labs

One job for two ready-made harnesses

```
claims desk/
├── claims/          to decide
├── policies/
├── customers/
├── policy_wording.md
├── claims_manual.md
├── company.json     the limits
└── decisions/       its output
```

1. **One instruction**
   Decide every claim and leave a note for the handler
2. **No tools from us**
   The harness reads, searches and writes the files itself
3. **Scored afterwards**
   Its decisions are checked against the handlers’ answers

Lab 1 runs it on the Claude Agent SDK and Lab 2 on Deep Agents

Lab 1: the Claude Agent SDK

In `labs/week1day5_lab1.ipynb` on Claude Sonnet 5.5

```
TOOLS = ["Read", "Glob", "Grep", "Write",
         "Task", "TaskCreate", "TaskUpdate"]

options = ClaudeAgentOptions(
    model="claude-sonnet-5-5",
    cwd=str(DESK),
    tools=TOOLS,
    agents={"claim-handler": claim_handler},
    max_budget_usd=1.00,
)

async for message in query(
        prompt="Please work the queue.",
        options=options):
    ...
```

Built-in file tools

Read, Glob, Grep and Write on the claims desk

A task list

TaskCreate and TaskUpdate keep the plan

A subagent

Task hands each claim to a fresh claim-handler

Watch-only unless you have an Anthropic key: five claims cost about 40 cents

Lab 2: Deep Agents

In `labs/week1day5_lab2.ipynb` on DeepSeek V4.1 Flash

```
llm = ChatOpenAI(
    model="deepseek/deepseek-v4.1-flash",
    base_url=OPENROUTER_BASE_URL,
    reasoning_effort="low",
)

agent = create_deep_agent(
    model=llm,
    system_prompt=INSTRUCTIONS,
    subagents=[claim_handler],
    middleware=[TodoListMiddleware()],
    backend=FilesystemBackend(
        root_dir=DESK, virtual_mode=True),
)
```

Built-in file tools
ls, read_file, grep and write_file on the claims desk

A to-do list
write_todos, from the TodoListMiddleware we add

A subagent
task hands each claim to a fresh claim-handler

Five claims cost about 3 cents. The free twin runs on a free model.

Upgrading the product

The same harness built by hand on create_agent



A to-do list

Stored in the backend and shown on the dashboard as the agent works



A helper agent for each claim

`work_claim` starts a fresh agent on one claim and gets back one line



The wording and the manual to look things up in

Tools that list the sections of a document and read one of them



Problems corrected automatically

A failed claim goes back in the queue and is tried again

One run now works the whole queue on DeepSeek V4.1 Flash

Building it with Claude Code

The change is written up in `docs/HARNESS.md`

1. **Ask Claude Code for a plan**

```
Please plan the harness in docs/HARNESS.md
```

2. **Let it build and commit each phase**

```
Please implement the plan for the harness
```

3. **Try it on five claims**

```
uv run xxx # however Claude Code implements this
```

4. **Or check out my version**

```
git checkout week1day5-end
```

Some logistics before we begin

Two paths you could take



Alternative 1: Keep your version

You'll have your version of UnderCover from Day 3

You need to copy `labs/week1day5_product_HARNESS.md` into `docs/HARNESS.md`

Your journey will be slightly different to mine, and that's all good!



Alternative 2: Switch to my version

Run `git checkout week1day5-start` to switch across to my version

If git mentions local changes, run `git stash` first

Then run `npm install`

You'll find `HARNESS.md` in the docs folder

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect

Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect

Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer AI ENGINEER

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course



33% Complete